

PROMELA Modeling

DJ

$n = 6$

Edges are nodes 0 and $n+1$

Nodes numbers 1- n

Left channels L : 0 - 5 : state from node L

Right channels R: 0- 5 : state from node R+2

Unbuffered Channels:

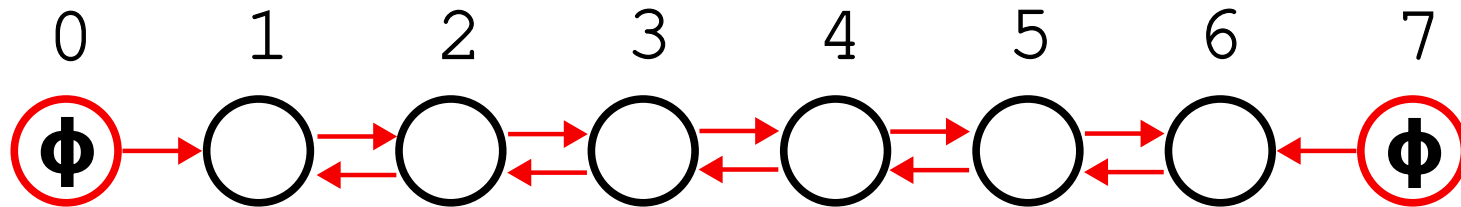
chan left[6] = [0] of {int,int}

chan right[6] = [0] of {int, int}

Buffered Channels:

chan left[6] = [10] of {int, int}

chan right[6] = [10] of {int, int}



Symbols on the channel are the tuple {state, index}

Index is used for completion assertion (Assert index=2 n -2 && state = F)

Index is the local node's notion of time in terms of transactions.

Rendezvous or Buffered Channels

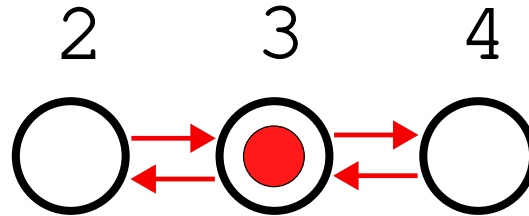
ϕ sends " ϕ " in a loop to signal edges

Nodes send their state left and right

Nodes read state from left and right

```
active proctype phi0() {
  int index;
  while (0==0)
    :: right[0] ! {0, index}
    :: index = index+1;
    od
}
```

```
active proctype phi1() {
  int index;
  do
    :: left[5] ! {0, index}
    :: index = index+1;
    od
}
```



```

active proctype node(n) {
  int state;
  int leftstate;
  int leftindex;
  int rightstate;
  int rightindex;
  int index;
  while (index < (2n-1))
    right[n-1] ? {leftstate, leftindex};
    left[n-2] ? {rightstate, rightindex};
    if
      :: (leftstate == rightstate == index) ->
        state = nextstatecalc(leftstate, rightstate, state);
        index = index + 1;
    fi
}

```

```

assert ((index < 2n-2) && (state != F)) || ((index == 2n-2) && (state == F))

```

- Wakesman 16 state

```
$ wakesman_fsp -p <width> #generate promela
```

```
$ wakesman_fsp -t #generate tables
```

```
$ wakesman_fsp -r <width> # Run
```

```
$ wakeman_fsp -s # Generate sverilog
```

- Mazoyer 6 state

```
$ mazoyer_fsp -p <width> #generate promela
```

```
$ mazoyer_fsp -t #generate tables
```

```
$ mazoyer_fsp -r <width> # Run
```

```
$ mazoyer_fsp -s # Generate sverilog
```

Command Line Tools

Graph FSP

- TBD

SPIN Results

- TBD